

Assignment 4

2026/04/09

The goal of this assignment is to familiarize yourself with some concepts in network security. This assignment mostly involves writing small Python programs for each question. For this assignment, you will have to refer to the Python API documentation for the secure sockets layer to help you write the programs: <https://docs.python.org/3/library/ssl.html>. This assignment is intended to give you another taste of networking in the real world, e.g., figuring out how to use a network API and implementing a new network protocol (the Diffie-Hellman key exchange). We use the terms SSL and TLS interchangeably in this assignment.

This assignment can be performed on your local machine and it can also be completed in GCP VM. We recommend you use GCP VM.

Submission

Your submission to “Assignment 4” should consist of 5 files (2 coding files and 2 trace files and 1 report file):

```
ssl_web_client.py
ssl_web_server.py
ssl_trace.txt
non_ssl_trace.txt
report.txt
```

1 Implementing SSL Web Client and “Server” (20 Points)

For this part of the assignment, we will use the Python ssl and socket libraries to implement a basic web server and client, both of which can be enabled to use TLS (and therefore serve/receive HTTPS rather than HTTP traffic). Starter code is provided in `ssl_web_client.py` and `ssl_web_server.py`.

The actual code you will write is relatively small. You will instead use these programs to answer some questions about TLS and Public Key Infrastructure. Part of these questions will also involve deploying your web server on a VM, assigning it a domain and obtaining a real, usable certificate for your server.

1.1 HTTP(S) Background

Although we have not discussed HTTP much in this course, you should be somewhat familiar with it given its use on the internet. Obviously, there are many technical details that go into HTTP, but for this assignment we will implement a bare minimum HTTP client and server, such that they are interoperable with existing websites and browsers, respectively.

Therefore, all you need to know about HTTP is that it is a text based protocol run over TCP (of course, in recent development, HTTP/3 has switched to running on UDP). Client requests and server responses consist of a header with a defined format, followed by some content separated from the header by new lines. A client connects to a TCP server and sends the request as text over the TCP socket, and the server replies with the

response and closes the socket.¹ The starter code provides some examples of well formed HTTP requests and responses that you can use.

HTTPS is simply HTTP run over a TCP socket secured with TLS.

HTTP and HTTPS by default run on ports 80 and 443, and websites will expect HTTP connections on these. However, you can run HTTP(S) servers on arbitrary ports if desired.

1.2 Implementation

The starter code is relatively structured, with a few spots annotated with TODOs. Please make sure that your code keeps the same command line arguments. However, you are also free to add additional command line arguments, for example to deal with self signed certificates.

We recommended implementing things in the following order.

1. First get `ssl_web_client.py` to work with SSL disabled. This should be relatively easy since you should have experience implementing TCP sockets from assignment 1. You can test your implementation against any website, or on localhost by using the `netcat` tool.
2. Next, implement wrapping the client socket with an SSL socket (if the `-ssl` flag is set). Again, you will need to refer to the Python ssl library: <https://docs.python.org/3/library/ssl.html>. You can now test your client against existing websites that support HTTPS. Remember to change the port if SSL is enabled!

Also, as described in the starter code, you need to implement having your client print the certificate provided by the server (*hint: use `getpeercert()`*)

At this point, your client is complete, and you should be able to complete the questions associated with Part 2 of this assignment. You can now begin implementing the server.

3. Again, first get `ssl_web_server.py` to work with SSL disabled. You should to make sure the server works with your client (with SSL also disabled). You may also want to test your server with a browser.

Note, by default `ssl_web_server.py` runs on port 8000. Browsers support specifying a specific port like so: `http://localhost:8000`. Note you can replace `localhost` with the IP of you Cloud VM if you are using one.

4. Finally, implement wrapping connections with a SSL socket if the `-ssl` flag is set. Note your server will need a certificate; see Part 3 of the assignment for details. Make sure your implementation is compatible with your client. When testing your SSL server with a browser, also be aware that the URL needs to start with `https://` instead of `http://`.

We will run some basic automated tests with your client and server, however the majority of the assignment will consist of completing the questions in the following sections.

2 Surfing the web with SSL (20 points)

Let's use your SSL enabled web to explore some websites. Note, your client should read at least 1024 bytes of response from the server. If you have written your program correctly, you should have output that looks like the output below.

```
$ python3 ssl_web_client.py www.example.com --ssl --port 443
```

```
{'OCSP': ('http://ocsp.digicert.com',),
```

¹Nowadays, clients and servers will keep a connection open to facilitate multiple requests without the overhead of establishing a new connection. But again, we are doing the bare minimum for this assignment!

```
'caIssuers': ('http://cacerts.digicert.com/DigiCertGlobalG2TLSRSA2562020CA1-1.crt',),
'crlDistributionPoints': ('http://crl3.digicert.com/DigiCertGlobalG2TLSRSA2562020CA1-1.crl',
                          'http://crl4.digicert.com/DigiCertGlobalG2TLSRSA2562020CA1-1.crl'),
'issuer': (((('countryName', 'US'),),
              (('organizationName', 'DigiCert Inc'),),
              (('commonName', 'DigiCert Global G2 TLS RSA SHA256 2020 CA1'),)),),
'notAfter': 'Mar 1 23:59:59 2025 GMT',
'notBefore': 'Jan 30 00:00:00 2024 GMT',
'serialNumber': '075BCEF30689C8ADDF13E51AF4AFE187',
'subject': (((('countryName', 'US'),),
              (('stateOrProvinceName', 'California'),),
              (('localityName', 'Los Angeles'),),
              (('organizationName',
                'Internet\\xa0Corporation\\xa0for\\xa0Assigned\\xa0Names\\xa0and\\xa0'
                'Numbers'),),
              (('commonName', 'www.example.org'),)),),
'subjectAltName': (('DNS', 'www.example.org'),
                  ('DNS', 'example.net'),
                  ('DNS', 'example.edu'),
                  ('DNS', 'example.com'),
                  ('DNS', 'example.org'),
                  ('DNS', 'www.example.com'),
                  ('DNS', 'www.example.edu'),
                  ('DNS', 'www.example.net')),
'version': 3}
```

===== HTTP Response =====

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Age: 465362
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Fri, 12 Apr 2024 19:17:24 GMT
Etag: "3147526947"
Expires: Fri, 19 Apr 2024 19:17:24 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECS (nyd/D151)
Vary: Accept-Encoding
X-Cache: HIT
Content-Length: 1256
```

Note, depending on the website and how you implemented your client (specifically the specific `recv()` calls) you may only see the HTTP header and not the website content. One example of a website where the solution client would see the website body is news.ycombinator.com/.

3 Deploying the Web Server (20 Points)

Running an SSL server requires creating a certificate, which then needs to be signed by a certificate authority. To start we'll use a self-signed certificate, where you yourself will certify that you are who you say you are. This is obviously not secure and not recommended for practical use. To create a self-signed certificate, you can use the instructions provided here: <https://docs.python.org/3/library/ssl.html#certificates>.

Because SSL enforces good security practices by default, this won't work out of the box because the client will reject the server's self-signed certificate as invalid. To override this setting, you will have to use the following Python snippet when creating the SSL context for both your client and your server.

```
context.check_hostname=False
```

```
context.verify_mode=ssl.CERT_NONE
```

You should be able at this point to run your server and client against each other without any errors.

```
python3 ssl_web_server.py --address 0.0.0.0 --port PORT --ssl
    --cert-file <path-to-cert>
```

You can choose any unoccupied ports. e.g.,

```
python3 ssl_web_server.py --address 0.0.0.0 --port 8000 --ssl
    --cert-file cert.pem
```

cert.pem is what I have generated based on the instructions from <https://docs.python.org/3/library/ssl.html#certificates>.

On the client side, you run

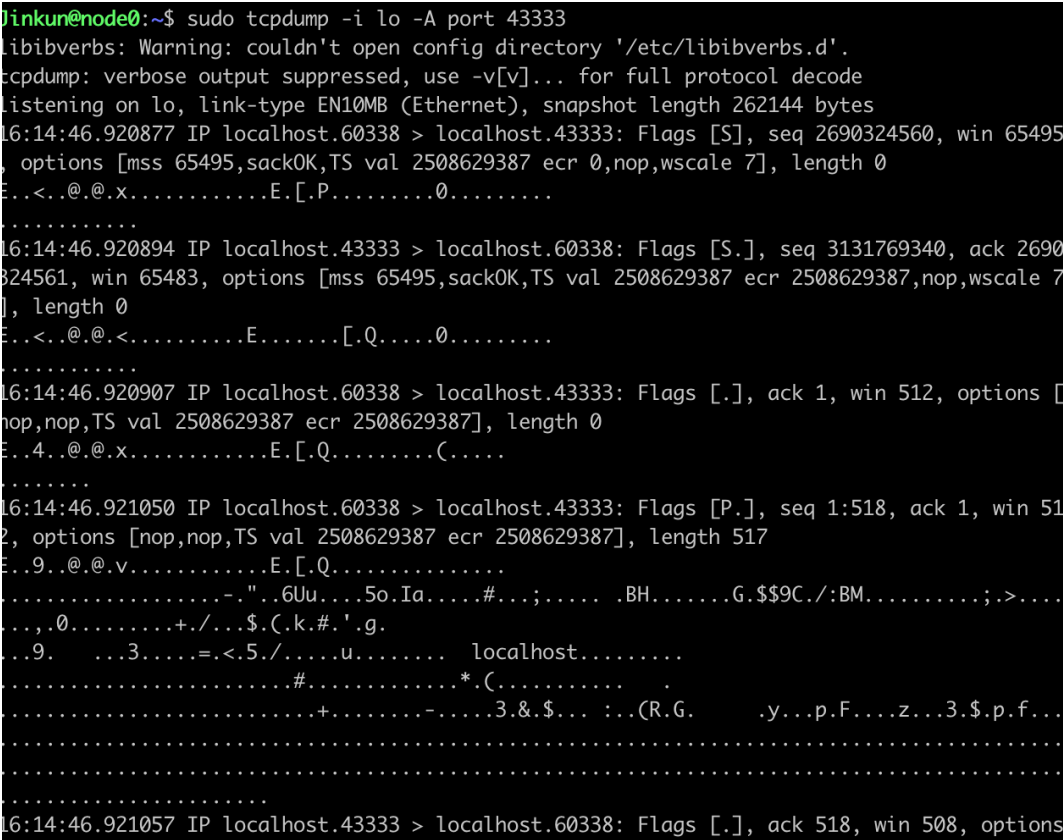
```
python3 ssl_web_client.py localhost --ssl --port 8000
```

4 Inspecting SSL Traffic (40 Points)

Once you have the SSL client and server working with each other run the following command to observe the packets on port 8000 (or other port of your choosing) over the loopback interface (i.e. localhost traffic).

```
sudo tcpdump -i lo -A port 8000
```

Here we show an example trace in Figure 1



```
linkun@node0:~$ sudo tcpdump -i lo -A port 43333
libibverbs: Warning: couldn't open config directory '/etc/libibverbs.d'.
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
16:14:46.920877 IP localhost.60338 > localhost.43333: Flags [S], seq 2690324560, win 65495
, options [mss 65495,sackOK,TS val 2508629387 ecr 0,nop,wscale 7], length 0
E..<..@.x.....E.[.P.....0.....
.....
16:14:46.920894 IP localhost.43333 > localhost.60338: Flags [S.], seq 3131769340, ack 2690
324561, win 65483, options [mss 65495,sackOK,TS val 2508629387 ecr 2508629387,nop,wscale 7
], length 0
E..<..@.<.....E.....[.Q....0.....
.....
16:14:46.920907 IP localhost.60338 > localhost.43333: Flags [.], ack 1, win 512, options [
nop,nop,TS val 2508629387 ecr 2508629387], length 0
E..4..@.x.....E.[.Q.....C.....
.....
16:14:46.921050 IP localhost.60338 > localhost.43333: Flags [P.], seq 1:518, ack 1, win 51
2, options [nop,nop,TS val 2508629387 ecr 2508629387], length 517
E..9..@.v.....E.[.Q.....
.....
.....-".6Uu...5o.Ia....#...;.....BH.....G.$9C./:BM.....;.>....
...0.....+./...$.C.k.#.'g.
...9....3.....=<.5./.....u.....localhost.....
.....#.....*.C.....
.....+.....-.....3.&.$... :.(R.G.....y...p.F....z...3.$p.f...
.....
.....
16:14:46.921057 IP localhost.43333 > localhost.60338: Flags [.], ack 518, win 508, options
```

Figure 1: Example Trace captured by tcpdump

Then run your client and server over localhost on port 8000 **without ssl enabled** (removing `-ssl` and `-cert-file`).

How many packets are exchanged?

Then repeat the experiment **with ssl enabled**. How many packets are exchanged now?

You should capture the packet traces for both run, and save the traces as `non_ssl_trace.txt` and `ssl_trace.txt`, respectively.

What is the reason for the change in number of packets, and do you see any other differences?

Include your answers to these questions in `report.txt`